



Checkpoint

- 7.27 What header file must you `#include` in order to define vector objects?
- 7.28 Write a definition statement for a vector named `frogs`. `frogs` should be an empty vector of `ints`.
- 7.29 Write a definition statement for a vector named `lizards`. `lizards` should be a vector of 20 `floats`.
- 7.30 Write a definition statement for a vector named `toads`. `toads` should be a vector of 100 `chars`, with each element initialized to `'Z'`.
- 7.31 `gators` is an empty vector of `ints`. Write a statement that stores the value 27 in `gators`.
- 7.32 `snakes` is a vector of `doubles`, with 10 elements. Write a statement that stores the value 12.897 in element 4 of the `snakes` vector.

Review Questions and Exercises

Short Answer

1. What is the difference between a size declarator and a subscript?
2. Look at the following array definition:


```
int values[10];
```

 How many elements does the array have?
 What is the subscript of the first element in the array?
 What is the subscript of the last element in the array?
 Assuming that an `int` uses 4 bytes of memory, how much memory does the array use?
3. Why should a function that accepts an array as an argument, and processes that array, also accept an argument specifying the array's size?
4. Consider the following array definition:


```
int values[5] = { 4, 7, 6, 8, 2 };
```

 What does each of the following statements display?

```
cout << values[4] << endl; _____
```

```
cout << (values[2] + values[3]) << endl; _____
```

```
cout << ++values[1] << endl; _____
```
5. How can you declare an integer array of size 10 and initialize values of all elements to 0?
6. Look at the following array definition:


```
int numbers[5] = { 1, 2, 3 };
```

 What value is stored in `numbers[2]`?
 What value is stored in `numbers[4]`?

7. Assuming that `array1` and `array2` are both arrays, why is it not possible to assign the contents of `array2` to `array1` with the following statement?

```
array1 = array2;
```
8. Assuming that `numbers` is an array of `doubles`, will the following statement display the contents of the array?

```
cout << numbers << endl;
```
9. Is an array passed to a function by value or by reference?
10. When you pass an array name as an argument to a function, what is actually being passed?
11. How do you establish a parallel relationship between two or more arrays?
12. Look at the following array definition:

```
double sales[8][10];
```

 How many rows does the array have?
 How many columns does the array have?
 How many elements does the array have?
 Write a statement that stores a number in the last column of the last row in the array.
13. When writing a function that accepts a two-dimensional array as an argument, which size declarator must you provide in the parameter for the array?
14. What advantages does a `vector` offer over an array?

Fill-in-the-Blank

15. The _____ indicates the number of elements, or values, an array can hold.
16. The size declarator must be a(n) _____ with a value greater than _____.
17. The _____ of an element in the array is denoted by its subscript.
18. The maximum subscript number of an array of size declarator `N` is _____.
19. The number inside the brackets of an array definition is the _____, but the number inside an array's brackets in an assignment statement, or any other statement that works with the contents of the array, is the _____.
20. The size declarator of an array can be a(n) _____ or a(n) _____ but it can never be a variable.
21. The 5th element in an array `somearray[]` is stored in _____.
22. If an array is partially initialized, the uninitialized elements will be set to _____.
23. If the size declarator of an array definition is omitted, C++ counts the number of items in the _____ to determine how large the array should be.
24. If an array is declared as `int myarray[10]`, the memory size of the array will be _____ bytes.

25. You cannot use the _____ operator to copy data from one array to another in a single statement.
26. Any time the name of an array is used without brackets and a subscript, it is seen as _____.
27. To pass an array to a function, pass the _____ of the array.
28. A(n) _____ array is like several arrays of the same type put together.
29. It's best to think of a two-dimensional array as having _____ and _____.
30. The number of size declarators in an n-dimensional array is _____.
31. When a two-dimensional array `my2Darray[6][5]` is declared, it implies that the array has _____ rows with _____ elements in each.
32. When a two-dimensional array is passed to a function, the _____ size must be specified.
33. The _____ is a collection of programmer-defined data types and algorithms that you may use in your programs.
34. The two types of containers defined by the STL are _____ and _____.
35. The `vector` data type is a(n) _____ container.
36. To define a `vector` in your program, you must `#include` the _____ header file.
37. To store a value in a `vector` that does not have a starting size, or that is already full, use the _____ member function.
38. To determine the number of elements in a `vector`, use the _____ member function.
39. Use the _____ member function to remove the last element from a `vector`.
40. To completely clear the contents of a `vector`, use the _____ member function.

Algorithm Workbench

41. Define a floating point array `myvalues` with 20 elements. Write a `for` loop that assigns the first 20 odd numbers to the array elements.
42. The numeric arrays `NumArray1` and `NumArray2` have 50 elements. Write a `for` loop to find `SumArray` where each element is the sum of the corresponding elements of the two arrays.
43. In a program, you need to store the identification numbers of ten employees (as `ints`) and their weekly gross pay (as `doubles`).
 - A) Define two arrays that may be used in parallel to store the ten employee identification numbers and gross pay amounts.
 - B) Write a loop that uses these arrays to print each employee's identification number and weekly gross pay.
44. Define a two-dimensional array of integers named `Matrix`, where both the number of rows and the number of columns are equal to 50.

45. In a program, you need to store the populations of 12 countries.
- A) Define two arrays that may be used in parallel to store the names of the countries and their populations.
 - B) Write a loop that uses these arrays to print each country's name and its population.
46. The following code totals the values in two arrays: `numberArray1` and `numberArray2`. Both arrays have 25 elements. Will the code print the correct sum of values for both arrays? Why or why not?

```
int total = 0;           // Accumulator
int count;              // Loop counter
// Calculate and display the total of the first array.
for (count = 0; count < 24; count++)
    total += numberArray1[count];
cout << "The total for numberArray1 is " << total << endl;
// Calculate and display the total of the second array.
for (count = 0; count < 24; count++)
    total += numberArray2[count];
cout << "The total for numberArray2 is " << total << endl;
```

47. Look at the following array definition:
- ```
int array[10][5];
```
- Write a statement that assigns 100 to the last column of the first row of this array.
- Write a for loop that assigns 10 to all the elements of the first column.
48. `myMatrix` is a two-dimensional array with 10 rows and 10 columns. Write code to find the sum of the elements of its main diagonal from top-left to bottom-right.
49. An application uses a two-dimensional array defined as follows:
- ```
int values[10][20];
```
- Write code to find the minimum element of the first row.
- Write code to find the maximum element of the last column.

True or False

50. T F An array may contain elements with varied data types.
51. T F To calculate the amount of memory used by an array, multiply the number of elements by the number of bytes each element uses.
52. T F The individual elements of an array are accessed and indexed by unique numbers.
53. T F The subscript of an array may be a floating-point number.
54. T F If the value of a subscript or index is greater than the largest valid subscript of an array, the compiler will not report any error.
55. T F The elements in an array may be stored in non-contiguous locations in the main memory.
56. T F Arrays may have subscripts that have both positive and negative values.
57. T F You can write programs that use invalid subscripts for an array.

- 58. T F Arrays cannot be initialized when they are defined. A loop or other means must be used.
- 59. T F The values in an initialization list are stored in the array in the order they appear in the list.
- 60. T F A constant array must always have an initialization list.
- 61. T F If an array is partially initialized, it is possible to initialize the last few elements of the array without initializing the first portion.
- 62. T F If you leave an element uninitialized, you do not have to leave all the ones that follow it uninitialized.
- 63. T F If you leave out the size declarator of an array definition, you do not have to include an initialization list.
- 64. T F The uninitialized elements of a `string` array will automatically be set to the value `"0"`.
- 65. T F You cannot use the assignment operator to copy one array's contents to another in a single statement.
- 66. T F When an array name is used without brackets and a subscript, it is seen as the value of the first element in the array.
- 67. T F To pass an array to a function, pass the name of the array.
- 68. T F When defining a parameter variable to hold a single-dimensional array argument, you do not have to include the size declarator.
- 69. T F When an array is passed to a function, the function has access to the original array.
- 70. T F A two-dimensional array is like several identical arrays put together.
- 71. T F It's best to think of two-dimensional arrays as having rows and columns.
- 72. T F The first size declarator (in the declaration of a two-dimensional array) represents the number of columns. The second size definition represents the number of rows.
- 73. T F In a two-dimensional array defined as `myarray[10][20]`, the last element is accessed using the syntax `myarray[9][19]`.
- 74. T F C++ allows you to create arrays with three or more dimensions.
- 75. T F A `vector` is an associative container.
- 76. T F To use a `vector`, you must include the `vector` header file.
- 77. T F `vectors` can report the number of elements they contain.
- 78. T F You can use the `[]` operator to insert a value into a `vector` that has no elements.
- 79. T F If you add a value to a `vector` that is already full, the `vector` will automatically increase its size to accommodate the new value.

Find the Errors

Each of the following definitions and program segments has errors. Locate as many as you can.

- ```
80. int size = 10;
 double array[size];

81. int collection[-20];

82. int values[5];
 for (int i =1 ; i<=5 ; i++)
 {
 values[i] = 2 * i;
 cout << "Values :" << values[i];
 }

83. int array[4] = {2.3, 4.6, 7.8, 9.0, 6.7};

84. double series[] = { 2.3, "qwe", 2, '$', 44.3};

85. int array [4][] = {{1,2},{3,4},{5,6},{7,8}};

86. char greeting[] = {'H', 'e', 'l', 'l', 'o'};
 cout << greeting;

87. int array1[4], array2[4] = {3, 6, 9, 12};
 array1 = array2;

88. void showValues(int nums)
 {
 for (int count = 0; count < 8; count++)
 cout << nums[count];
 }

89. void showValues(int nums[4][])
 {
 for (rows = 0; rows < 4; rows++)
 for (cols = 0; cols < 5; cols++)
 cout << nums[rows][cols];
 }

90. vector<int> numbers = { 1, 2, 3, 4 };
```

### Programming Challenges

#### 1. Largest and Second Largest Even Numbers

Write a program to input 15 integers into an array. The program should then display the largest and second largest even numbers in the array.

#### 2. Rainfall Statistics

Write a program that lets the user enter the total rainfall for each of 12 months into an array of doubles. The program should calculate and display the total rainfall for the year, the average monthly rainfall, and the months with the highest and lowest amounts.

*Input Validation: Do not accept negative numbers for monthly rainfall figures.*



**VideoNote**  
**Solving the**  
**Chips and**  
**Salsa Problem**

### 3. Chips and Salsa

Write a program that lets a maker of chips and salsa keep track of sales for five different types of salsa: mild, medium, sweet, hot, and zesty. The program should use two parallel 5-element arrays: an array of strings that holds the five salsa names, and an array of integers that holds the number of jars sold during the past month for each salsa type. The salsa names should be stored using an initialization list at the time the name array is created. The program should prompt the user to enter the number of jars sold for each type. Once this sales data has been entered, the program should produce a report that displays sales for each salsa type, total sales, and the names of the highest selling and lowest selling products.

*Input Validation: Do not accept negative values for number of jars sold.*

### 4. More Positives Than Negatives

Write a function that accepts two arguments: an array of integers and its size. The function should return a Boolean indicating whether the array contains more positive numbers than negative numbers, not including zeros. Test the function in a program.

### 5. Conference Sessions

An upcoming conference about C/C++ programming has three sessions planned:

- A session on the new features of C++ 17
- A session on functional programming in C/C++
- A session on lambda functions

Attendees can subscribe for any session. The organizers want to keep track of which attendees are attending which session.

Write a program that stores this information in a two-dimensional array of Booleans, where each row represents an attendee and each column represents their subscription for a session. You can restrict the program to seven attendees, resulting in a  $7 \times 3$  array. The program should first have the user input the data for each attendee. Write two functions that take the array as their argument. One function should print out the number of people in each session and the other should return the number of people who subscribed to all sessions.

### 6. Temperature Variation

The weather service center of a certain city records the highest temperature, lowest temperature, and humidity of each day. Write a program that maintains three arrays and initializes the highest temperature, lowest temperature, and humidity in the three arrays, for each day of the first 10 days of a month. The program should then create a weather report that includes the following information:

- Highest temperature and lowest temperature.
- Diurnal variation in each day.
- Humidity of each day and whether it is greater or less than the average humidity.

Finally, in an end note, it should display the maximum temperature, minimum temperature, and overall variation in temperature in the duration.

### 7. Union of Two Sets

A set is a collection of similar elements. A union of two sets A and B is a set of elements in which each element belongs to set A, set B, or both. It is represented as  $A \cup B$ . For example, if  $A = \{1, 3, 5, 7\}$  and  $B = \{3, 6, 7, 9, 11\}$ , then  $A \cup B = \{1, 3, 5, 6, 7, 9, 11\}$ .

Write a program that represents set A, set B, and set  $A \cup B$  as three arrays. The maximum size of the arrays representing sets A and B should be 10. The program should ask the user to input the sizes of sets A and B and then the elements of sets A and B. It should then display  $A \cup B$ .

*Input Validation: Do not accept values less than 1 or more than 10 as sizes of sets A and B.*

8. Lo Shu Magic Square

The Lo Shu Magic Square is a grid with 3 rows and 3 columns shown in Figure 7-19. The Lo Shu Magic Square has the following properties:

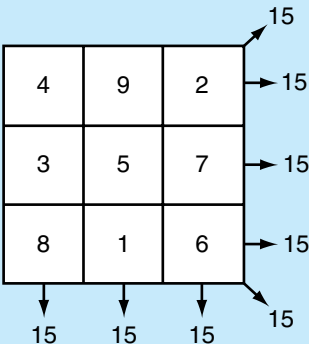
- The grid contains the numbers 1 through 9 exactly.
- The sum of each row, each column, and each diagonal all add up to the same number. This is shown in Figure 7-20.

In a program, you can simulate a magic square using a two-dimensional array. Write a function that accepts a two-dimensional array as an argument, and determines whether the array is a Lo Shu Magic Square. Test the function in a program.

Figure 7-19 Lo Shu Magic Square

|   |   |   |
|---|---|---|
| 4 | 9 | 2 |
| 3 | 5 | 7 |
| 8 | 1 | 6 |

Figure 7-20 Sums of the rows, columns, and diagonals



9. Price Calculation

Write a program that uses the following arrays:

- plu: an array of price lookup codes (PLU) for products at a grocery store
- price: an array of gross prices of products
- vat: an array of the Value Added Tax (VAT) rate for each product

The program should relate the data in each array through the subscripts. The  $n$ th element of each array contains, respectively, the PLU, the gross price, and the VAT rate of the same product. For example, element 0 in the `price` array should contain the gross price of the product whose PLU code is stored in element 0 of the `plu` array. The same product's VAT rate should be stored in element 0 of the `vat` array. Write a function that takes three arguments—the array `plu`, the length of the array, and the PLU of a specific product—and returns the index of the PLU in the array. If the PLU cannot be found, the function must return `-1`.

Write another function that takes six arguments: the three arrays, the length of these arrays (they have the same length), a PLU code, and the number of articles. The function should return the net total for this number of products, and `-1` if the PLU code does not exist.

#### 10. Driver's License Exam

The local Driver's License Office has asked you to write a program that grades the written portion of the driver's license exam. The exam has 20 multiple-choice questions. Here are the correct answers:

- |      |       |       |       |
|------|-------|-------|-------|
| 1. A | 6. B  | 11. A | 16. C |
| 2. D | 7. A  | 12. C | 17. C |
| 3. B | 8. B  | 13. D | 18. A |
| 4. B | 9. C  | 14. B | 19. D |
| 5. C | 10. D | 15. D | 20. B |

Your program should store the correct answers shown above in an array. It should ask the user to enter the student's answers for each of the 20 questions, and the answers should be stored in another array. After the student's answers have been entered, the program should display a message indicating whether the student passed or failed the exam. (A student must correctly answer 15 of the 20 questions to pass the exam.) It should then display the total number of correctly answered questions, the total number of incorrectly answered questions, and a list showing the question numbers of the incorrectly answered questions.

*Input Validation: Only accept the letters A, B, C, or D as answers.*

#### 11. Exam Grader

One of your professors has asked you to write a program to grade her final exams, which consist of only 20 multiple-choice questions. Each question has one of four possible answers: A, B, C, or D. The file `CorrectAnswers.txt` contains the correct answers for all of the questions, with each answer written on a separate line. The first line contains the answer to the first question, the second line contains the answer to the second question, and so forth. (Download the book's source code from the Computer Science Portal at [www.pearsonglobaleditions.com/gaddis](http://www.pearsonglobaleditions.com/gaddis). You will find the file in the Chapter 07 folder.)

Write a program that reads the contents of the `CorrectAnswers.txt` file into a `char` array, then reads the contents of another file, containing a student's answers, into a second `char` array. (You can use the file `StudentAnswers.txt` for testing purposes. This

file is also in the Chapter 07 source code folder.) The program should determine the number of questions that the student missed, then display the following:

- A list of the questions missed by the student, showing the correct answer and the incorrect answer provided by the student for each missed question
- The total number of questions missed
- The percentage of questions answered correctly. This can be calculated as  
$$\text{Correctly Answered Questions} \div \text{Total Number of Questions}$$
- If the percentage of correctly answered questions is 70 percent or greater, the program should indicate that the student passed the exam. Otherwise, it should indicate that the student failed the exam.

12. A Graded Social Network

The goal of a social network is to connect people. In an early attempt, people were asked to grade each other on a scale of 1 to 10. An algorithm would decide who was friend with whom. The idea was that anyone whom someone graded a 5 or more would be that person’s friend.

In a certain experiment with three people, this was the input:

| Person | Friend Score<br>of Person 1 | Friend Score<br>of Person 2 | Friend Score<br>of Person 3 |
|--------|-----------------------------|-----------------------------|-----------------------------|
| John   | 10                          | 4                           | 7                           |
| Mike   | 3                           | 2                           | 4                           |
| Mary   | 10                          | 10                          | 10                          |

It was expected that people would be each other’s friends, but it turned out that this was not the case. In this case, for example, Mike indicated that he had no friends at all. Because of this and various other obvious reasons, this social network failed. However, it may still be interesting to write a program to repeat the experiment. The program should use a `string` array of length 5 to hold the names of the people in the network, and five `int` arrays to hold their friend scores.

The program should let the user enter five names and five scores per person. It should then display the number of friends per person and their names.

13. Social Network Modification

Modify the solution of Programming Challenge 12 such that two people can only be considered friends if each gives the other person a friend score of at least 5.

14. Lottery Application

Write a program that simulates a lottery. The program should have an array of five integers named `lottery` and should generate a random number in the range of 0 through 9 for each element in the array. The user should enter five digits, which should be stored in an integer array named `user`. The program is to compare the corresponding elements in the two arrays and keep a count of the digits that match. For example, the following shows the `lottery` array and the `user` array with sample numbers stored in each. There are two matching digits (elements 2 and 4).

Lottery array:

|   |   |   |   |   |
|---|---|---|---|---|
| 7 | 4 | 9 | 1 | 3 |
|---|---|---|---|---|

User array:

|   |   |   |   |   |
|---|---|---|---|---|
| 4 | 2 | 9 | 7 | 3 |
|---|---|---|---|---|

The program should display the random numbers stored in the `lottery` array and the number of matching digits. If all of the digits match, display a message proclaiming the user as a grand prize winner.

### 15. vector Modification

Modify the National Commerce Bank case study presented in Program 7-23 so `pin1`, `pin2`, and `pin3` are vectors instead of arrays. You must also modify the `testPIN` function to accept a vector instead of an array.

### 16. World Series Champions

If you have downloaded this book's source code, you will find the following files in this chapter's folder:

- `Teams.txt`—This file contains a list of several Major League baseball teams in alphabetical order. Each team listed in the file has won the World Series at least once.
- `WorldSeriesWinners.txt`—This file contains a chronological list of the World Series' winning teams from 1903 to 2012. (The first line in the file is the name of the team that won in 1903, and the last line is the name of the team that won in 2012. Note the World Series was not played in 1904 or 1994.)

Write a program that displays the contents of the `Teams.txt` file on the screen and prompts the user to enter the name of one of the teams. The program should then display the number of times that team has won the World Series in the time period from 1903 to 2012.



**TIP:** Read the contents of the `WorldSeriesWinners.txt` file into an array or vector. When the user enters the name of a team, the program should step through the array or vector counting the number of times the selected team appears.

### 17. Name Search

If you have downloaded this book's source code, you will find the following files in this chapter's folder:

- `GirlNames.txt`—This file contains a list of the 200 most popular names given to girls born in the United States from 2000 to 2009.
- `BoyNames.txt`—This file contains a list of the 200 most popular names given to boys born in the United States from 2000 to 2009.

Write a program that reads the contents of the two files into two separate arrays or vectors. The user should be able to enter a boy's name, a girl's name, or both, and the application should display messages indicating whether the names were among the most popular.

**18. Tic-Tac-Toe Game**

Write a program that allows two players to play a game of tic-tac-toe. Use a two-dimensional char array with three rows and three columns as the game board. Each element of the array should be initialized with an asterisk (\*). The program should run a loop that does the following:

- Displays the contents of the board array.
- Allows player 1 to select a location on the board for an X. The program should ask the user to enter the row and column numbers.
- Allows player 2 to select a location on the board for an O. The program should ask the user to enter the row and column numbers.
- Determines whether a player has won, or a tie has occurred. If a player has won, the program should declare that player the winner and end. If a tie has occurred, the program should display an appropriate message and end.

Player 1 wins when there are three Xs in a row on the game board. The Xs can appear in a row, in a column, or diagonally across the board. Player 2 wins when there are three Os in a row on the game board. The Os can appear in a row, in a column, or diagonally across the board. A tie occurs when all of the locations on the board are full, but there is no winner.

**19. Hot or Cold Hints**

Write a program in which the user must guess a random number in the range from 1 to 100. The user must keep on guessing until the number is correctly guessed, but encouragements are printed to make sure that the user keeps on playing. In the student sample programs for this book, you will find a text file named `19_hints.txt`. The file contains eight responses, such as “That is very cold. Try again.”, “You are getting closer.”, “I can feel the heat.”, and so forth. The program should read the responses from the file into an array or vector. It should prompt the user to enter a value. If the value is correct, the program should end, but if the guess is not correct, it should be compared to the previous attempt. If this attempt is worse than the previous guess, one of the first four responses should be displayed. Otherwise, display one of the last four responses. For the first guess, the first four responses should be used when the difference between the guess and the correct answer is larger than 20.

*Contents of 19\_hints.txt:*

```
Incorrect and very cold. You are not getting closer.
Oh my! You are far away and getting cold.
It's chilly in here. No number in the neighborhood.
It seems like winter is settling in. The secret is well hidden.
Not good yet, but you are getting closer and warmer.
Almost there, I can already feel the heat.
You are reaching the goal. It already feels comfy and warm.
Better than ever. The temperature is increasing.
```

**20. 1994 Gas Prices**

In the student sample programs for this book, you will find a text file named `1994_Weekly_Gas_Averages.txt`. The file contains the average gas price for each week in the year 1994 (There are 52 lines in the file. Line 1 contains the average price for week 1, line 2 contains the average price for week 2, and so forth.) Write a program that reads the gas prices from the file into an array or a vector. The program should do the following:

- Display the lowest average price of the year, along with the week number for that price, and the name of the month in which it occurred.

- Display the highest average price of the year, along with the week number for that price, and the name of the month in which it occurred.
- Display the average gas price for each month. (To get the average price for a given month, calculate the average of the average weekly prices for that month.)

## 21. 2D Array Operations

Write a program that creates a two-dimensional array initialized with test data. Use any data type you wish. The program should have the following functions:

- **getTotal**—This function should accept a two-dimensional array as its argument and return the total of all the values in the array.
- **getAverage**—This function should accept a two-dimensional array as its argument and return the average of all the values in the array.
- **getRowTotal**—This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The function should return the total of the values in the specified row.
- **getColumnTotal**—This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a column in the array. The function should return the total of the values in the specified column.
- **getHighestInRow**—This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The function should return the highest value in the specified row of the array.
- **getLowestInRow**—This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The function should return the lowest value in the specified row of the array.

Demonstrate each of the functions in this program.

## Group Project

### 22. Theater Seating

This program should be designed and written by a team of students. Here are some suggestions:

- One student should design function `main`, which will call the other functions in the program. The remainder of the functions will be designed by other members of the team.
- The requirements of the program should be analyzed so that each student is given about the same workload.
- The parameters and return types of each function should be decided in advance.
- The program can be implemented as a multi-file program, or all the functions can be cut and pasted into the main file.

Here is the assignment: Write a program that can be used by a small theater to sell tickets for performances. The theater's auditorium has 15 rows of seats, with 30 seats in each row. The program should display a screen that shows which seats are available and which are taken. For example, the following screen shows a chart depicting each

seat in the theater. Seats that are taken are represented by an \* symbol, and seats that are available are represented by a # symbol.

| Seats  |                                |
|--------|--------------------------------|
|        | 123456789012345678901234567890 |
| Row 1  | ***###**###*#####**##          |
| Row 2  | #####**#####**                 |
| Row 3  | **###**#####**###              |
| Row 4  | **#####**#####                 |
| Row 5  | *****#####                     |
| Row 6  | #####**#####                   |
| Row 7  | #####**#####                   |
| Row 8  | *****##**#####                 |
| Row 9  | #####**#####                   |
| Row 10 | #####**#####                   |
| Row 11 | #*****#####*                   |
| Row 12 | #####**#####*                  |
| Row 13 | ###*****#####*                 |
| Row 14 | #####                          |
| Row 15 | #####                          |

Here is a list of tasks this program must perform:

- When the program begins, it should ask the user to enter the seat prices for each row. The prices can be stored in a separate array. (Alternatively, the prices may be read from a file.)
- Once the prices are entered, the program should display a seating chart similar to the one shown above. The user may enter the row and seat numbers for tickets being sold. Every time a ticket or group of tickets is purchased, the program should display the total ticket prices and update the seating chart.
- The program should keep a total of all ticket sales. The user should be given an option of viewing this amount.
- The program should also give the user an option to see a list of how many seats have been sold, how many seats are available in each row, and how many seats are available in the entire auditorium.

*Input Validation: When tickets are being sold, do not accept row or seat numbers that do not exist. When someone requests a particular seat, the program should make sure that seat is available before it is sold.*